Learned Index Structures: Recursive Model Index vs. B-Tree

CS631 course project, checkpoint 1
Paper: Kraska, Beutel, Chi, Dean and Polyzotis, *The Case for Learned Index Structures*, SIGMOD 2018
Team: Nischal (23b1024) Rithvik (23b0939)

1. What we are doing

The paper treats an index as a function from a key to its position in a sorted array. That function is just the CDF of the keys scaled by N, so a model can learn it. We are reproducing the range index part of the paper (Section 3): a two-stage Recursive Model Index (RMI) that predicts where a key is, compared against a read-only B-Tree on lookup time and index size.

2. What we have implemented

Everything is in C++17 (about 2,000 lines) and runs on synthetic data from 10M to 200M keys.

The RMI (`src/rmi.hpp`, `src/rmi.cpp`). Stage 1 is a single linear model that sends a key to one of K stage-2 models. We fit it twice, once on the key and once on log(key), and keep whichever spreads the keys more evenly over the K models. Each stage-2 model is a straight line fitted by least squares to its own keys. Keys go up to 2^{63}, so the fit subtracts the means first and sums in `long double`; the plain formula lost too much precision.

Each stage-2 model also stores the largest errors it made on its own keys, above and below. A lookup only binary-searches between `pred + min_err` and `pred + max_err`, which means a key that is in the data is always found. Stage 1 never decreases as the key grows, so each stage-2 model owns one contiguous slice of the array. We store where each slice starts instead of copying keys, which is what lets us train on 200M keys.

The B-Tree baseline (`src/btree.hpp`, `src/btree.cpp`). It is built bottom-up with every page full, for page sizes 32, 64, 128 and 256. The sorted array is the leaf level, and each level above it is one cache-aligned array of keys. Because pages are full, the child of slot j in node i is node i·P + j, so we store no pointers. We wanted a strong baseline, and this one is smaller and faster than a textbook B-Tree.

The benchmark (`bench/benchmark.cpp`). Both indexes are in the same program, built with the same flags, and use the same branch-free binary search. Before timing anything we look up every key and check its position, and check that 1,000 keys not in the data return "not found". A failure stops the run. Each timing is the median of 5 runs of 1M random lookups. We also have a separate RMI program (`rmi/`) that reports error statistics up to 200M keys and tries exponential search.

Setup. Three datasets: dense (0 to N−1), uniform_sparse (random 64-bit keys) and lognormal (the paper's skewed synthetic data). N is 10M and 100M (up to 200M for the RMI statistics) and K is 1k, 10k or 100k. All runs are on an Intel i7-13620H with g++ 13.3 and `-O3 -march=native`.

3. Results so far

All 69 configurations we ran (42 RMI and B-Tree comparisons, 27 RMI-only runs) found every key at its exact position and returned no false positives. At 100M keys, compared with a B-Tree using 128-key pages:

Dataset (N = 100M)	B-Tree, 128-key pages	Best RMI	Speedup	Index size
dense	367 ns, 6.0 MB	K=1k: 33 ns, 23 KB	11.1×	262× smaller
uniform_sparse	373 ns, 6.0 MB	K=100k: 111 ns, 2.3 MB	3.4×	2.6× smaller

lognormal	377 ns, 6.0 MB	K=100k: 196 ns, 2.3 MB	1.9×	2.6× smaller

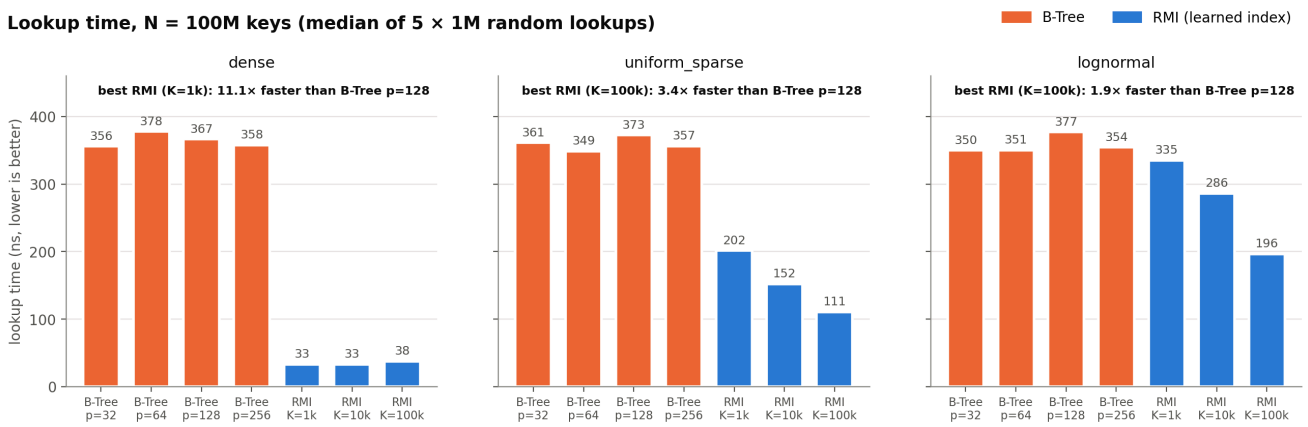


Figure 1. Lookup time at N = 100M (B-Tree in orange, RMI in blue). Every RMI configuration we tried was faster than every B-Tree page size, at both 10M and 100M keys.

Some of what we saw surprised us:

- Running the two RMI models costs 4 to 15 ns, only 2 to 14% of a lookup. Most of the time goes into the final search waiting on main memory. The B-Tree spends about 80 ns just getting from the root to a leaf.
- On dense data the model's error is zero, yet 29 of the 33 ns is still spent fetching that one key from RAM. A perfect model removes the comparisons, but it can't remove the memory access.
- Lognormal is where the RMI struggles. Adding models brings the average search window down from 7,425 to 33 positions at 100M, but the worst model stays at about 1.46M positions at 200M no matter what K is. A straight line in stage 1 can't follow lognormal's S-shaped CDF.
- The size advantage depends on N. The RMI's size is set by K, while the B-Tree grows with the data. At 10M keys the K=100k RMI is actually 3.8× larger than the B-Tree, so the fastest RMI isn't always the smallest one.

4. What we plan to do next

Feature	Paper	Why
Small neural network (or multivariate model) for stage 1	§3.2	Learn the lognormal curve and fix the worst models
Hybrid index (Algorithm 1)	§3.3	Replace stage-2 models whose error is too large with small B-Trees
Model-biased binary search and biased quaternary search	§3.4	Start searching at the predicted position
A real dataset (SOSD: books, osm or fb)	§3.7.1	The paper's Weblogs and Maps data are not public
K = 10k, 50k, 100k and 200k	§3.7.1	Use the same stage-2 sizes as the paper

If time allows, we would also like to try learned hash maps (Section 4) and learned Bloom filters (Section 5).